Uber Drive Pulse || Design Document

Chirag|| Divyam Nagpal || Nevica Gupta

1. Product Vision

User Persona

The primary users of **Uber Drive Pulse** are **gig-economy drivers**, particularly those working on ride-hailing platforms like Uber. These drivers spend long hours on the road and depend on efficient trip management and safe driving behaviour to maximise earnings.

Typical characteristics of the user include:

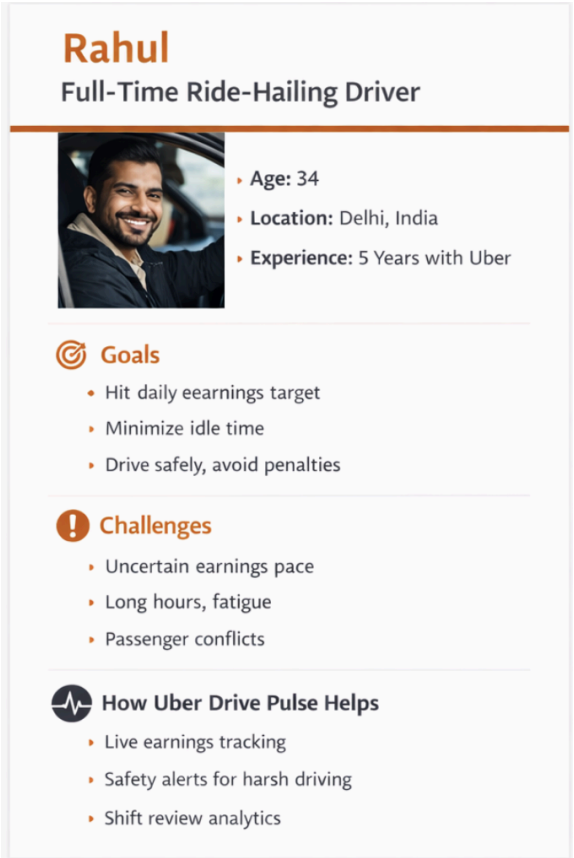
- Works extended shifts during peak hours
- Needs to track earnings relative to daily targets
- Operates in dynamic urban environments
- Has limited attention available for complex interfaces while driving

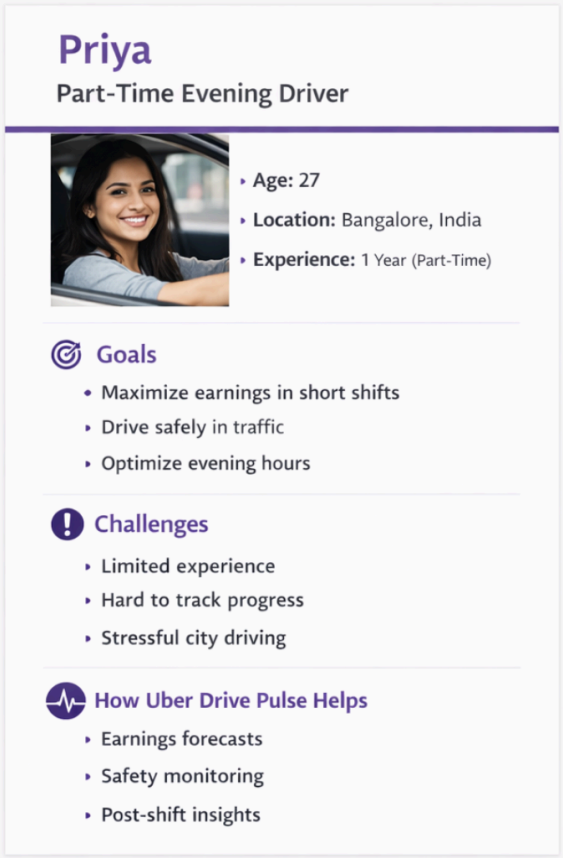
Drivers often face two key uncertainties during their shift:

1. **Safety awareness**: recognising unsafe driving patterns or stressful cabin situations.
2. **Earnings visibility**: understanding whether their current driving pace will help them achieve their target earnings.

Uber Drive Pulse is designed to provide **real-time insights into both safety and earnings performance**, enabling drivers to make informed decisions during their shifts.

User Persona Profiles:





Core Value Proposition

Uber Drive Pulse provides two major benefits:

Real-Time Safety Awareness

The system analyses telemetry signals such as motion and audio to detect potential safety issues, including:

- Harsh braking
- Sudden acceleration spikes
- Excessive cabin noise
- Passenger conflict scenarios

These signals are processed in the Safety Engine implemented in *src/safety_engine.py.*

Financial Performance Monitoring

Drivers often operate with a **daily or shift-based earnings goal**.

Uber Drive Pulse continuously tracks:

- Current earnings
- Earnings velocity
- Forecast relative to the target goal
- Remaining progress required

These calculations are performed in the Financial Engine implemented in *src/financial_engine.py.*

Glanceable Interface Design (Mobile-First)

Drivers typically interact with the system through a **mobile device mounted on the dashboard**, meaning the interface must be **quick to interpret and minimally distracting**.

Therefore, Uber Drive Pulse is designed to satisfy the **"glanceable interface" principle**, enabling drivers to understand their status in less than 2 **seconds**.

The interface implementation is in *src/app.py*, which builds the Streamlit dashboard.

Minimal Cognitive Load

The **Live Shift view** shows only essential metrics:

- Current earnings
- Earnings velocity
- Forecast status
- Goal progress bar

By limiting the number of elements displayed, the driver can **quickly** interpret their status without distraction.

Colour-Based Status Indicators

The dashboard uses clear colour-coded signals:

- Green → Ahead
- Blue → On Track
- Orange → Behind
- Gold → Goal Achieved

These visual cues allow drivers to quickly understand their performance without having to read complex numerical data.

Separation of Live vs Analytical Views

The dashboard provides two interfaces implemented in *src/app.py:*

Live Shift View

- Displays minimal real-time information
- Designed for in-drive usage

Pit Stop Analytics

- Provides deeper post-shift insights
- Intended for review during breaks or after the shift

This separation ensures the system remains safe and usable while driving.

2. Stress Detection Algorithm

The **stress detection system** analyses telemetry streams to identify potentially dangerous or stressful driving situations.

The logic is implemented in *src/safety_engine.py.*

The system processes two primary sensor streams:

- Accelerometer motion data
- Cabin audio intensity

These streams are generated using the synthetic data generator located in: *src/generate_synthetic_data.py.*

Synthetic Dataset Scope

To simulate realistic driver behaviour and telemetry signals, the system uses a synthetic dataset generated using *src/generate_synthetic_data.py.*

For the MVP prototype, the dataset represents:

- 1 synthetic driver

- 9 trips occurring within a single shift/day
- Per-second accelerometer telemetry
- Per-second audio intensity telemetry

The generated datasets are stored in the ***data/*** directory, including:

- drivers.csv: synthetic driver profile
- driver_goals.csv: driver earnings target
- trips.csv: trip-level earnings and timestamps
- accelerometer_data.csv: motion telemetry stream
- audio_intensity_data.csv: cabin audio intensity stream

Each trip contains injected anomalies designed to test the detection logic implemented in *src/safety_engine.py,* such as harsh braking events, device drops, loud cabin noise, and passenger conflict signals.

This controlled synthetic setup enables deterministic testing of both the Safety Engine (*src/safety_engine.py*) and the Financial Engine (*src/financial_engine.py*).

Motion Signal Processing

Accelerometer data provides readings across three axes: “ax”, “ay”, “az”.

Raw accelerometer signals are often noisy due to:

- Road surface vibrations
- Sensor jitter
- Environmental disturbances

To handle this, the safety engine performs signal processing before event detection.

Noise Cleaning

- The system first removes gravitational acceleration to isolate actual vehicle motion: **linear_accel = sqrt($ax^2 + ay^2 + (az - g)^2$)**
- This calculation is implemented in *src/safety_engine.py* during motion event processing.

Sliding Window Smoothing

- Instead of reacting to a single spike, the system maintains a **sliding window buffer of recent motion signals**. This buffer is implemented using deque-based rolling windows in *src/safety_engine.py.*
- The engine computes average acceleration across this window to smooth noise and avoid false alerts.

Speed-Aware Filtering

- Some motion spikes occur when the vehicle is stationary (for example, closing a door).
- To reduce false positives, harsh braking detection is triggered only when **vehicle speed exceeds a minimum threshold**.
- Speed information is derived from the accelerometer dataset in data/accelerometer_data.csv, processed in *src/safety_engine.py.*

Audio Signal Processing

Cabin sound intensity is analysed to detect abnormal conditions such as:

- Loud music
- Passenger arguments
- Sudden noise spikes

The system processes **audio amplitude values rather than raw audio recordings**, preserving privacy.

Audio processing is implemented in:

- src/safety_engine.py
- using telemetry generated by:
- data/audio_intensity_data.csv

Audio Window Averaging

Audio signals are aggregated across a short time window:

average_audio_intensity = mean(audio_window)

This helps detect **sustained abnormal noise levels** rather than temporary spikes.

Multi-Sensor Fusion

- Single sensor signals can produce false positives.
- To increase reliability, the system correlates signals across multiple sensors.
- Example detection rule: Harsh Braking + Sustained High Audio = Passenger Conflict Event

  This fusion logic is implemented in *src/safety_engine.py* and allows the system to detect higher-severity events.

Safety Event Logging

All detected safety events are written to *data/flagged_moments.csv.*

Each event includes:

- Timestamp
- Trip ID
- Event type
- Severity level
- Explanation

These logs are later visualised in the dashboard through *src/app.py.*

3. Earnings Velocity Algorithm

The **financial engine** tracks the driver's earning performance during the shift.

- The logic for this component is implemented in: *src/financial_engine.py.*
- The engine processes **trip completion events** generated in: *data/trips.csv* and updates financial progress metrics.

Current Earnings Speed

- The driver's earning speed is calculated using: **earnings_velocity = cumulative_earnings / elapsed_shift_hours**
- This calculation is implemented in *src/financial_engine.py.*
- The cumulative earnings and trip data are updated as the simulator processes events.

Forecasting Progress

- The system compares the driver's current earnings velocity with the **required velocity needed to reach the earnings goal**.
- Required velocity is calculated using: **required_velocity = earnings_goal / shift_duration**
- Driver goals are loaded from: *data/driver_goals.csv* and processed within *src/financial_engine.py.*

Forecast Status Classification

Based on the comparison between actual and required velocity, the system assigns one of four states:

Status	Meaning
AHEAD	The driver is earning faster than the required pace
ON_TRACK	Driver near the target pace
BEHIND	The driver needs a higher pace
ACHIEVED	Target earnings reached

These statuses are written to *data/earnings_velocity_log.csv* and visualised in *src/app.py.*

Handling the Cold Start Problem

- At the beginning of a shift, there is insufficient data to estimate earnings velocity accurately. This is known as the **cold start problem**. To address this, the system initially estimates earnings pace using **trip-based projections** rather than time-based velocity.
- During the first few trips, the algorithm estimates pace using: **average_trip_earning × expected_trip_frequency**
- This logic is implemented in *src/financial_engine.py.*
- Once sufficient data accumulates, the algorithm transitions to the standard velocity calculation.

4. Event Simulation Pipeline

The system simulates a real-time streaming architecture using an event loop implemented in *src/simulator.py.*

The simulator performs the following steps:

1. Load telemetry datasets from the ***data/*** directory

2. Merge audio and motion streams
3. Sort events chronologically
4. Dispatch events to the appropriate engine

Event routing:

Motion events → safety_engine.py
Audio events → safety_engine.py
Trip completion → financial_engine.py

This architecture mimics a real-world telemetry event processing pipeline.

5. Execution Strategy

MVP Scope

For demonstration purposes, the MVP simulates telemetry data for **a single driver completing 9 trips during one driving shift**, allowing the event pipeline and analytics engines to be tested in a controlled environment.

The MVP demonstrates a working telemetry analytics pipeline, including:

- Synthetic telemetry generation (*generate_synthetic_data.py*)
- Event-driven simulator (*simulator.py*)
- Safety detection engine (*safety_engine.py*)
- Financial analytics engine (*financial_engine.py*)
- Live dashboard (*app.py*)

Together, these components simulate how a **real-time driver analytics system could operate in production**.

Phased Rollout Plan

Phase 1: Motion Monitoring

- Integrate smartphone accelerometer APIs
- Detect harsh braking and aggressive driving

Phase 2: Cabin Environment Monitoring

- Integrate audio telemetry
- Detect conflict situations

Phase 3: Financial Intelligence

- Real-time earnings forecasting
- Dynamic shift pacing guidance

Phase 4: Fleet-Level Monitoring

- Safety scoring across drivers
- Fleet analytics dashboards

Cut Line (Scope Decisions)

Due to time constraints during the hackathon, the following features were intentionally excluded from the MVP.

Machine Learning Models

The system uses **deterministic heuristics instead of machine learning**.

This decision allowed:

- Faster development
- Fully explainable detection logic
- Lower computational requirements

Real Sensor Integration

The MVP uses **synthetic telemetry data generated in** *generate_synthetic_data.py* rather than real sensors.

Future versions could integrate:

- Smartphone accelerometer APIs
- Vehicle telemetry
- Microphone sensors

Streaming Infrastructure

Instead of production streaming platforms like Kafka or Pub/Sub in Redis, the MVP simulates streaming using:

- Event loops (*simulator.py*)
- Structured log files (*CSV*)

This allowed rapid prototyping while still demonstrating a realistic system architecture.

Conclusion

Uber Drive Pulse demonstrates how **real-time telemetry analytics and financial intelligence** can improve the safety and productivity of gig-economy drivers.

By combining:

- Sensor signal processing
- Earnings forecasting
- A mobile-friendly glanceable interface

The system enables drivers to:

- Monitor safety conditions
- Track earnings progress
- Make informed decisions during their shifts.